

Spring Boot Homework

Use this homework after completing Spring Boot classroom notes. The goal is to make students practice definitions, code, debugging, and interview-style explanation.

Submission Rules

1. Create a separate Spring Boot project for coding homework.
2. Use proper package structure: controller, service, model, dto, exception.
3. Write clean class names and method names.
4. Test every API using browser or Postman.
5. Take screenshots of API responses.
6. Write short answers in your own words.

Homework 1: Spring Boot Basics

Theory Questions

Question 1:

What problem does Spring Boot solve?

Question 2:

Write four differences between Spring Framework and Spring Boot.

Question 3:

Explain this line:
Spring Boot = Spring + auto setup + embedded server + starters

Question 4:

What is the use of embedded Tomcat?

Question 5:

What is the difference between JAR deployment and WAR deployment?

Practical Task

Create a Spring Boot project using Spring Initializr.

Use:

```
Project: Maven
Language: Java
Packaging: Jar
Dependency: Spring Web
```

Create one API:

```
GET /welcome
```

Expected response:

```
Welcome to Spring Boot Homework
```

Homework 2: Main Class And Component Scanning

Theory Questions

Question 1:

```
What is @SpringBootApplication?
```

Question 2:

```
Which three annotations are internally included in @SpringBootApplication?
```

Question 3:

```
Why should main class be placed in the root package?
```

Question 4:

```
What happens if controller class is outside the main package?
```

Practical Task

Create this package structure:

```
com.example.demo
  DemoApplication
  controller
    HelloController
  service
    HelloService
```

Create:

```
GET /scan-demo
```

Expected response:

Component scanning is working

Then intentionally move `HelloService` outside the root package and observe the error.

Write the error message in your notebook.

Homework 3: REST Controller And GET API

Theory Questions

Question 1:

What is `@RestController`?

Question 2:

What is `@GetMapping`?

Question 3:

Who calls controller methods in Spring Boot?

Question 4:

Explain this flow:
Client -> Tomcat -> DispatcherServlet -> Controller -> Response

Practical Task

Create `StudentController`.

Create these APIs:

```
GET /students/message
GET /students/names
GET /students/count
```

Expected responses:

```
/students/message -> Welcome Student
/students/names   -> Rahul, Priya, Amit
/students/count   -> 3
```

Homework 4: Returning Object As JSON

Theory Questions

Question 1:

How does Spring Boot convert Java object into JSON?

Question 2:

What is Jackson?

Question 3:

Why are getters important when returning object as JSON?

Practical Task

Create model class:

```
Student
id
name
course
marks
```

Create API:

GET /students/one

Expected JSON:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "Java",
  "marks": 85
}
```

Create another API:

GET /students/all

Return list of at least three students.

Homework 5: PathVariable And RequestParam

Theory Questions

Question 1:

What is @PathVariable?

Question 2:

What is @RequestParam?

Question 3:

When should we use PathVariable and when should we use RequestParam?

Practical Task

Create these APIs:

```
GET /students/{id}
GET /students/search?course=java
GET /students/filter?minMarks=80
GET /students/page?page=0&size=5
```

Expected:

```
/students/101 should return student id 101.
/students/search?course=java should search by course.
/students/filter?minMarks=80 should return students with marks greater than or equal to 80.
/students/page?page=0&size=5 should print page and size values.
```

Homework 6: POST API And RequestBody

Theory Questions

Question 1:

What is POST used for?

Question 2:

What is @RequestBody?

Question 3:

Why do we create DTO classes?

Practical Task

Create DTO:

```
StudentRequest
name
course
marks
```

Create API:

```
POST /students
```

Sample request:

```
{
  "name": "Priya",
  "course": "Spring Boot",
  "marks": 91
}
```

Expected response:

```
Student created: Priya
```

Homework 7: HTTP Methods

Theory Questions

Question 1:

```
Explain GET, POST, PUT, PATCH, DELETE.
```

Question 2:

```
What is the difference between PUT and PATCH?
```

Practical Task

Create these APIs:

```
GET /students
POST /students
PUT /students/{id}
PATCH /students/{id}/marks
DELETE /students/{id}
```

For now, return simple messages from each API.

Example:

```
Student updated with id: 101
```

Homework 8: ResponseEntity

Theory Questions

Question 1:

What is ResponseEntity?

Question 2:

Why should we return proper HTTP status codes?

Question 3:

What is wrong with returning "Student not found" with 200 OK?

Practical Task

Create:

```
GET /students/{id}
```

Rules:

```
If id is 101, return 200 OK with student data.  
If id is not 101, return 404 Not Found.  
If id is less than or equal to 0, return 400 Bad Request.
```

Homework 9: Layered Architecture

Theory Questions

Question 1:

What is layered architecture?

Question 2:

What is the responsibility of Controller?

Question 3:

What is the responsibility of Service?

Question 4:

What is the responsibility of Repository?

Practical Task

Create package structure:

```
controller
service
model
dto
```

Create:

```
StudentController
StudentService
Student
StudentRequest
```

Controller should not contain business logic.

Controller should call service.

Create APIs:

```
GET /students
GET /students/{id}
POST /students
DELETE /students/{id}
```

Use `ArrayList` inside service for now.

Homework 10: Dependency Injection

Theory Questions

Question 1:

```
What is Dependency Injection?
```

Question 2:

```
What are three types of Dependency Injection?
```

Question 3:

```
Why is constructor injection recommended?
```

Question 4:

```
What is the problem with field injection?
```

Practical Task

Create:


```
StudentController
StudentService
```

Show all three injection styles:

```
Field injection
Setter injection
Constructor injection
```

Create three endpoints:

```
GET /di/field
GET /di/setter
GET /di/constructor
```

Each endpoint should call a service method and return a message.

Autowiring Internal Task

For each injection style, write:

```
Which type is Spring using to find the dependency?
Field injection -> by type first
Setter injection -> by type first
Constructor injection -> by type first
```

Add constructor logs in both classes:

```
public StudentService() {
    System.out.println("StudentService bean created");
}
```

```
public StudentController(StudentService studentService) {
    this.studentService = studentService;
    System.out.println("StudentController bean created and StudentService injected");
}
```

Write the startup log order in notebook.

Explain:

```
Why is StudentService created before StudentController?
```

Homework 11: Multiple Beans, Qualifier, Primary

Theory Questions

Question 1:

```
What happens if Spring finds multiple beans of the same type?
```

Question 2:

```
What is @Qualifier?
```

Question 3:

```
What is @Primary?
```

Question 4:

```
What is the difference between @Qualifier and @Primary?
```

Practical Task

Create interface:

```
PaymentService
```

Create implementations:

```
UpiPaymentService  
CardPaymentService
```

Create controller:

```
PaymentController
```

Create APIs:

```
GET /pay/upi  
GET /pay/card  
GET /pay/default
```

Use:

```
@Qualifier for UPI and Card  
@Primary for default payment service
```

Multiple Bean Internal Task

Write what Spring sees internally:

```
PaymentService type has two beans:  
1. upiPaymentService  
2. cardPaymentService
```

Then explain:

```
Why does Spring fail without @Qualifier or @Primary?
```

Homework 12: Auto-Configuration Internals And Condition Logs

Theory Questions

Question 1:

What is auto-configuration?

Question 2:

What is the difference between component scan and auto-configuration?

Question 3:

What is `AutoConfiguration.imports`?

Question 4:

What is `@ConditionalOnClass`?

Question 5:

What is `@ConditionalOnMissingBean`?

Question 6:

What is `@ConditionalOnProperty`?

Question 7:

What does positive match mean in Spring Boot condition report?

Question 8:

What does negative match mean in Spring Boot condition report?

Practical Task 1: Show Condition Report In Logs

Add this in `application.properties` :

`debug=true`

Restart application.

Find this section in console:

CONDITIONS EVALUATION REPORT

Write in notebook:

1. One positive match
2. One negative match
3. Why it matched or did not match

Expected understanding:

Positive match = condition true, auto-configuration applied.
Negative match = condition false, auto-configuration skipped.

Practical Task 2: Create Conditional Bean

Create class:

```
package com.example.demo.config;

public class EmailSender {

    public EmailSender() {
        System.out.println("EmailSender bean created");
    }

    public String sendEmail() {
        return "Email sent";
    }
}
```

Create config:

```
package com.example.demo.config;

import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class NotificationConfig {

    @Bean
    @ConditionalOnProperty(
        name = "notification.email.enabled",
        havingValue = "true"
    )
    public EmailSender emailSender() {
        return new EmailSender();
    }
}
```

Case 1:

```
notification.email.enabled=true
```

Expected:

```
EmailSender bean created
```

Case 2:

```
notification.email.enabled=false
```

Expected:

```
EmailSender bean is not created
```

Write explanation:

```
How does this prove that Spring Boot creates beans based on conditions?
```

Practical Task 3: Actuator Conditions

Add actuator dependency.

Expose conditions:

```
management.endpoints.web.exposure.include=health,info,conditions
```

Open:

```
GET /actuator/conditions
```

Write:

```
What information is shown in this endpoint?
```

Homework 13: application.properties And @Value

Theory Questions

Question 1:

```
What is application.properties?
```

Question 2:

```
Why should values not be hardcoded in Java classes?
```

Question 3:

```
What is @Value?
```

Practical Task

Add these properties:

```
app.institute.name=Genie Ashwani
app.course.name=Spring Boot
app.trainer.name=Ashwani
```

Create API:

```
GET /app-info
```

Expected response:

```
Institute: Genie Ashwani, Course: Spring Boot, Trainer: Ashwani
```

Homework 14: Profiles

Theory Questions

Question 1:

```
What are profiles?
```

Question 2:

```
Why do we need dev, test, and prod profiles?
```

Question 3:

```
How do we activate a profile?
```

Practical Task

Create files:

```
application-dev.properties
application-prod.properties
```

In dev:

```
app.environment=Development
server.port=8081
```

In prod:

```
app.environment=Production
server.port=9090
```

Create API:

```
GET /environment
```

Response should show active environment value.

Homework 15: Request Validation

Theory Questions

Question 1:

What is request validation?

Question 2:

What is @Valid?

Question 3:

Write use of @NotBlank, @Min, @Email, and @Size.

Practical Task

Add validation dependency:

spring-boot-starter-validation

Create DTO:

```
StudentRequest
name should not be blank
email should be valid
marks should be minimum 0
course should not be blank
```

Create API:

POST /students

Test with valid and invalid JSON.

Write the response for both cases.

Homework 16: Exception Handling

Theory Questions

Question 1:

What is global exception handling?

Question 2:

What is `@RestControllerAdvice`?

Question 3:

What is `@ExceptionHandler`?

Practical Task

Create custom exception:

`StudentNotFoundException`

Create global exception handler:

`GlobalExceptionHandler`

Rules:

If student id does not exist, throw `StudentNotFoundException`.
`GlobalExceptionHandler` should return 404 Not Found with message.

API:

GET `/students/{id}`

Homework 17: Logging

Theory Questions

Question 1:

Why should we use logs instead of `System.out.println`?

Question 2:

Write common log levels.

Question 3:

When should we use INFO, DEBUG, WARN, and ERROR?

Practical Task

Add logs in:

`StudentController`
`StudentService`
`GlobalExceptionHandler`

Log:

```
When API starts
When service method starts
When student is not found
When exception is handled
```

Homework 18: Actuator

Theory Questions

Question 1:

```
What is Actuator?
```

Question 2:

```
Why is Actuator useful in production?
```

Question 3:

```
Why should we expose only required actuator endpoints?
```

Practical Task

Add actuator dependency:

```
spring-boot-starter-actuator
```

Expose:

```
management.endpoints.web.exposure.include=health,info
```

Test:

```
GET /actuator/health
GET /actuator/info
```

Write both responses in notebook.

Homework 19: Mini Project

Create a complete Student Management API without database.

Required APIs

```
GET /students
GET /students/{id}
POST /students
PUT /students/{id}
PATCH /students/{id}/marks
DELETE /students/{id}
GET /students/search?course=java
GET /students/filter?minMarks=80
```

Required Concepts

```
Controller
Service
Model
DTO
Constructor injection
ResponseEntity
PathVariable
RequestParam
RequestBody
Validation
Global exception handling
Logging
application.properties
```

Expected Package Structure

```
com.example.studentapp
    StudentAppApplication
    controller
        StudentController
    service
        StudentService
    model
        Student
    dto
        StudentRequest
    exception
        StudentNotFoundException
        GlobalExceptionHandler
```

Interview Homework

Students must prepare spoken answers for these questions.

1. What is Spring Boot?
2. Why Spring Boot?
3. What is auto-configuration?
4. What are starters?
5. What is embedded server?
6. What is @SpringBootApplication?
7. What is component scanning?
8. What is @RestController?
9. What is @GetMapping?
10. Difference between PathVariable and RequestParam?
11. What is @RequestBody?
12. What is ResponseEntity?
13. What is Dependency Injection?
14. Which dependency injection is recommended?
15. What is @Qualifier?
16. What is @Primary?
17. What are profiles?
18. What is validation?
19. What is global exception handling?
20. What is Actuator?

Final Submission Checklist

- Project runs successfully.
- All APIs are tested.
- Screenshots are attached.
- Theory answers are written.
- Code follows layered architecture.
- No business logic is written inside controller.
- Constructor injection is used for required dependencies.
- Proper status codes are returned.
- Validation and exception handling are implemented.
- Logs are added in important places.